

MDTS4214_711_PS_6

Daniel D. Mondal

2026-02-25

Problem set 6: Model Selection

To demonstrate the application of subset selection methods

Consider the “Hitters” data. For loading the data you need to install ISLR library. We wish to predict a baseball player’s salary on the basis of various predictors. So first we need to choose an optimal model.

```
rm(list=ls())
set.seed(123)
library(ISLR)
library(leaps)
str(Hitters)

## 'data.frame': 322 obs. of 20 variables:
## $ AtBat : int 293 315 479 496 321 594 185 298 323 401 ...
## $ Hits : int 66 81 130 141 87 169 37 73 81 92 ...
## $ HmRun : int 1 7 18 20 10 4 1 0 6 17 ...
## $ Runs : int 30 24 66 65 39 74 23 24 26 49 ...
## $ RBI : int 29 38 72 78 42 51 8 24 32 66 ...
## $ Walks : int 14 39 76 37 30 35 21 7 8 65 ...
## $ Years : int 1 14 3 11 2 11 2 3 2 13 ...
## $ CAtBat : int 293 3449 1624 5628 396 4408 214 509 341 5206 ...
## $ CHits : int 66 835 457 1575 101 1133 42 108 86 1332 ...
## $ CHmRun : int 1 69 63 225 12 19 1 0 6 253 ...
## $ CRuns : int 30 321 224 828 48 501 30 41 32 784 ...
## $ CRBI : int 29 414 266 838 46 336 9 37 34 890 ...
## $ CWalks : int 14 375 263 354 33 194 24 12 8 866 ...
## $ League : Factor w/ 2 levels "A","N": 1 2 1 2 2 1 2 1 2 1 ...
## $ Division : Factor w/ 2 levels "E","W": 1 2 2 1 1 2 1 2 2 1 ...
## $ PutOuts : int 446 632 880 200 805 282 76 121 143 0 ...
## $ Assists : int 33 43 82 11 40 421 127 283 290 0 ...
## $ Errors : int 20 10 14 3 4 25 7 9 19 0 ...
## $ Salary : num NA 475 480 500 91.5 750 70 100 75 1100 ...
## $ NewLeague: Factor w/ 2 levels "A","N": 1 2 1 2 2 1 1 1 2 1 ...
```

1. Clean the data by removing all rows that have missing value in any variable. You will be working with this clean data.

```
hitter_clean = na.omit(Hitters)
head(hitter_clean)
```

```
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun
## -Alan Ashby    315   81    7   24  38   39   14   3449   835    69
## -Alvin Davis   479  130   18   66  72   76    3   1624   457    63
## -Andre Dawson  496  141   20   65  78   37   11   5628  1575   225
## -Andres Galarra 321   87   10   39  42   30    2    396   101    12
## -Alfredo Griffin 594  169    4   74  51   35   11   4408  1133    19
## -Al Newman    185   37    1   23   8   21    2    214   42     1
##           CRuns CRBI CWalks League Division PutOuts Assists Errors
## -Alan Ashby    321  414   375     N         W     632    43    10
## -Alvin Davis   224  266   263     A         W     880    82    14
## -Andre Dawson  828  838   354     N         E     200    11     3
## -Andres Galarra 48   46    33     N         E     805    40     4
## -Alfredo Griffin 501  336   194     A         W     282   421    25
## -Al Newman     30    9    24     N         E      76   127     7
##           Salary NewLeague
## -Alan Ashby    475.0      N
## -Alvin Davis   480.0      A
## -Andre Dawson  500.0      N
## -Andres Galarra 91.5      N
## -Alfredo Griffin 750.0     A
## -Al Newman     70.0      A
```

2. Split the data randomly into train-test keeping 80% as training sample and remaining as test sample.

```
n = nrow(hitter_clean)
train = sample(1:n, floor(0.8*n))
train_data = hitter_clean[train, ]
test_data = hitter_clean[-train, ]

head(train_data)

##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns
## -Marvell Wynne  288   76    7   34  37   15    4   1644   408    16   198
## -Bill Almon    196   43    7   29  27   30   13   3231   825    36   376
## -Rick Cerone   216   56    4   22  18   15   12   2796   665    43   266
## -Jack Howell   151   41    4   26  21   19    2    288    68     9    45
## -Spike Owen    528  122    1   67  45   51    4   1716   403    12   211
## -Tom Paciorek  213   61    4   17  22    3   17   4061  1145    83   488
##           CRBI CWalks League Division PutOuts Assists Errors Salary
## -Marvell Wynne  120   113     N         W     203     3     3    240
## -Bill Almon    290   238     N         E      80    45     8    240
## -Rick Cerone   304   198     A         E    391    44     4    250
## -Jack Howell    39    35     A         W     28    56     2     95
## -Spike Owen    146   155     A         W    209   372    17   350
## -Tom Paciorek  491   244     A         W    178    45     4    235
##           NewLeague
## -Marvell Wynne      N
## -Bill Almon         N
## -Rick Cerone        A
## -Jack Howell        A
## -Spike Owen         A
## -Tom Paciorek       A

head(test_data)
```

```
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns
## -Alvin Davis   479  130    18   66  72   76    3   1624   457    63   224
## -Andre Dawson  496  141    20   65  78   37   11   5628  1575   225   828
## -Andy VanSlyke 418  113    13   48  61   47    4   1512   392    41   205
## -Buddy Bell    568  158    20   89  75   73   15   8068  2273   177  1045
## -Barry Bonds   413   92    16   72  48   65    1   413    92    16    72
## -Bobby Bonilla 426  109     3   55  43   62    1   426   109     3    55
##           CRBI CWalks League Division PutOuts Assists Errors Salary
## -Alvin Davis   266   263      A      W      880     82    14   480
## -Andre Dawson  838   354      N      E     200     11     3   500
## -Andy VanSlyke 204   203      N      E     211     11     7   550
## -Buddy Bell    993   732      N      W     105    290    10   775
## -Barry Bonds    48    65      N      E     280      9     5   100
## -Bobby Bonilla  43    62      A      W     361     22     2   115
##           NewLeague
## -Alvin Davis      A
## -Andre Dawson      N
## -Andy VanSlyke      N
## -Buddy Bell        N
## -Barry Bonds        N
## -Bobby Bonilla      N
```

```
attach(train_data)
```

3. Perform the best subset selection on the training sample. While choosing the best model fit up to a 19 variable model. Choose the best model according to adjusted R^2 , BIC, AIC and C_p values. In case the best model differs according to the selection criteria, report the best fitted model according to each selection criterion. In case the best model differs according to the selection criteria, report the best fitted model according to each selection criterion.

```
reg1 = regsubsets(Salary ~ ., data = train_data, nvmax = 19)
reg1_sum = summary(reg1)
```

Best Subset selection according Aadjusted R^2 .

```
c_adjR2 = coef(reg1, which.max(reg1_sum$adjR2))
c_adjR2
```

```
## (Intercept)           AtBat           Hits           Walks           CAtBat           CRuns
## 154.1594365   -1.8722264     5.7547106     6.3852096    -0.2092154     1.7712871
##           CRBI           CWalks           DivisionW           PutOuts           Assists           Errors
## 1.1419024   -0.8989348   -121.8633686     0.2013298     0.5363804    -6.4726568
## NewLeagueN
## 75.8378982
```

```
predictors = c(names(c_adjR2)[-c(1,9,13)], "Division", "NewLeague")
predictors
```

```
## [1] "AtBat"      "Hits"       "Walks"      "CAtBat"     "CRuns"      "CRBI"
## [7] "CWalks"     "PutOuts"    "Assists"    "Errors"     "Division"    "NewLeagu
## e"
```

```

formula = reformulate(termlabels = predictors, response = "Salary")
formula

## Salary ~ AtBat + Hits + Walks + CAtBat + CRuns + CRBI + CWalks +
##      PutOuts + Assists + Errors + Division + NewLeague

m1 = lm(formula, train_data)

y_h1 = predict(m1)
y_test1 = predict(m1, newdata = test_data)

train_mse1 = mean((train_data$Salary - y_h1)^2)
train_mse1

## [1] 88935.8

test_mse1 = mean((test_data$Salary - y_test1)^2)
test_mse1

## [1] 130066.9

```

Best Subset selection according BIC

```

c_bic = coef(reg1, which.min(reg1_sum$bic))
c_bic

## (Intercept)      Walks      CAtBat      CHits      CHmRun  DivisionW
## 149.5853608    5.9582020   -0.5241611    2.0687418    1.7652811   -129.0165012

predictors = c(names(c_bic)[-c(1,6)], "Division")
predictors

## [1] "Walks"      "CAtBat"     "CHits"      "CHmRun"     "Division"

formula = reformulate(termlabels = predictors, response = "Salary")
formula

## Salary ~ Walks + CAtBat + CHits + CHmRun + Division

m2 = lm(formula, train_data)

y_h2 = predict(m2)
y_test2 = predict(m2, newdata = test_data)

train_mse2 = mean((train_data$Salary - y_h2)^2)
train_mse2

## [1] 97026.43

test_mse2 = mean((test_data$Salary - y_test2)^2)
test_mse2

## [1] 153090.5

```

Best Subset selection according Mallow's C_p measure:

```
c_cp = coef(reg1,which.min(reg1_sum$cp))
c_cp

## (Intercept)      AtBat      Hits      Walks      CAtBat      CRuns
## 134.1593585 -1.9328699  5.8271814  6.5037245 -0.2063323  1.7820180
##          CRBI      CWalks  DivisionW      PutOuts      Assists  NewLeagueN
##   1.1120229 -0.8879140 -121.2135304   0.1925628   0.3316973  71.1433129

predictors = c(names(c_cp)[-c(1,9,12)],"Division","NewLeague")
predictors

## [1] "AtBat"      "Hits"      "Walks"      "CAtBat"      "CRuns"      "CRBI"
## [7] "CWalks"     "PutOuts"   "Assists"    "Division"    "NewLeague"

formula = reformulate(termlabels = predictors, response = "Salary")
formula

## Salary ~ AtBat + Hits + Walks + CAtBat + CRuns + CRBI + CWalks +
## PutOuts + Assists + Division + NewLeague

m3 = lm(formula, train_data)

y_h3 = predict(m3)
y_test3 = predict(m3,newdata = test_data)

train_mse3 = mean((train_data$Salary - y_h3)^2)
train_mse3

## [1] 89790.63

test_mse3 = mean((test_data$Salary - y_test3)^2)
test_mse3

## [1] 125652.3
```

4. Find the training MSE and test MSE for each of the best models selected according to different criterion as mentioned in 3 and compare their values.

```
Measure = c("Adjusted R sq.", "BIC", "Mallow's Cp")
Train_MSE_1 = c(train_mse1, train_mse2, train_mse3)
Test_MSE_1 = c(test_mse1, test_mse2, test_mse3)

#Comparison
output = data.frame(Measure, Train_MSE_1, Test_MSE_1)
output

##      Measure Train_MSE_1 Test_MSE_1
## 1 Adjusted R sq.    88935.80  130066.9
## 2          BIC     97026.43  153090.5
## 3 Mallow's Cp     89790.63  125652.3
```

5. Repeat 3-4 using forward stepwise selection method.

FORWARD STEPWISE SELECTION

```
reg2 = regsubsets(Salary ~ ., data = train_data, method = "forward", nvmax = 19)
reg2_sum = summary(reg2)
```

Forward Subset selection according Aadjusted R^2 .

```
c_adjr2 = coef(reg2, which.max(reg2_sum$adjr2))
c_adjr2

## (Intercept)      AtBat      Hits      Walks      Years      CAtBat
## 207.8865937 -1.6603271  4.9552022  6.0893338 -12.4996617 -0.2504738
##      CHits      CRuns      CRBI      CWalks  DivisionW      PutOuts
##  0.3954684  1.4147636  1.0607231 -0.7225505 -123.4055931  0.1923965
##      Assists      Errors  NewLeagueN
##  0.5131110 -7.0115407  70.8774554

predictors = c(names(c_adjr2)[-c(1,11,15)], "Division", "NewLeague")
predictors

## [1] "AtBat"      "Hits"      "Walks"      "Years"      "CAtBat"      "CHits"
## [7] "CRuns"      "CRBI"      "CWalks"      "PutOuts"      "Assists"      "Errors"
## [13] "Division"    "NewLeague"

formula = reformulate(termlabels = predictors, response = "Salary")
formula

## Salary ~ AtBat + Hits + Walks + Years + CAtBat + CHits + CRuns +
##      CRBI + CWalks + PutOuts + Assists + Errors + Division + NewLeague

m1 = lm(formula, train_data)

y_h1 = predict(m1)
y_test1 = predict(m1, newdata = test_data)

train_mse1 = mean((train_data$Salary - y_h1)^2)
train_mse1

## [1] 88287.08

test_mse1 = mean((test_data$Salary - y_test1)^2)
test_mse1

## [1] 134001.7
```

Forward Subset selection according BIC

```
c_bic = coef(reg2, which.min(reg2_sum$bic))
c_bic
```

```
## (Intercept)      Walks      CAtBat      CHits      CRuns      CRBI
## 127.8159745    6.4746441   -0.4127407    1.1133040    1.0341530    0.8520825
##      CWalks    DivisionW
##   -0.5432536  -127.3877839

predictors = c(names(c_bic)[-c(1,8)], "Division")
predictors

## [1] "Walks"      "CAtBat"      "CHits"      "CRuns"      "CRBI"      "CWalks"      "Division"

formula = reformulate(termlabels = predictors, response = "Salary")
formula

## Salary ~ Walks + CAtBat + CHits + CRuns + CRBI + CWalks + Division

m2 = lm(formula, train_data)

y_h2 = predict(m2)
y_test2 = predict(m2, newdata = test_data)

train_mse2 = mean((train_data$Salary - y_h2)^2)
train_mse2

## [1] 95658.81

test_mse2 = mean((test_data$Salary - y_test2)^2)
test_mse2

## [1] 146540.9
```

Forward Subset selection according Mallow's C_p measure:

```
c_cp = coef(reg2, which.min(reg2_sum$cp))
c_cp

## (Intercept)      Walks      CAtBat      CHits      CRuns      CRBI
## 110.1015291    5.2753942   -0.4498891    1.0510348    1.3425171    0.9381944
##      CWalks    DivisionW      PutOuts      Assists      Errors    NewLeagueN
##   -0.5426157  -125.5159872    0.1609710    0.4662319   -7.6994411    74.6774479

predictors = c(names(c_cp)[-c(1,8,12)], "Division", "NewLeague")
predictors

## [1] "Walks"      "CAtBat"      "CHits"      "CRuns"      "CRBI"      "CWalks"
## [7] "PutOuts"     "Assists"     "Errors"     "Division"    "NewLeague"

formula = reformulate(termlabels = predictors, response = "Salary")
formula

## Salary ~ Walks + CAtBat + CHits + CRuns + CRBI + CWalks + PutOuts +
##      Assists + Errors + Division + NewLeague
```

```

m3 = lm(formula, train_data)

y_h3 = predict(m3)
y_test3 = predict(m3, newdata = test_data)

train_mse3 = mean((train_data$Salary - y_h3)^2)
train_mse3

## [1] 90888.44

test_mse3 = mean((test_data$Salary - y_test3)^2)
test_mse3

## [1] 145923.9

```

Comparison Matrix

```

Measure = c("Adjusted R sq.", "BIC", "Mallow's Cp")
Train_MSE_2 = c(train_mse1, train_mse2, train_mse3)
Test_MSE_2 = c(test_mse1, test_mse2, test_mse3)

#Comparison
output = data.frame(Measure, Train_MSE_2, Test_MSE_2)
output

##           Measure Train_MSE_2 Test_MSE_2
## 1 Adjusted R sq.    88287.08   134001.7
## 2              BIC    95658.81   146540.9
## 3 Mallow's Cp     90888.44   145923.9

```

BACKWARD STEPWISE SELECTION

```

reg3 = regsubsets(Salary ~ ., data = train_data, nvmax = 19, method = "backward")
reg3_sum = summary(reg3)

```

Backward Subset selection according Aadjusted R^2 .

```

c_adj2 = coef(reg3, which.max(reg3_sum$adj2))
c_adj2

## (Intercept)      AtBat      Hits      Walks      CAtBat      CHits
## 151.3709120 -1.5393497  4.6939494  6.1305798 -0.3519984  1.1424820
##      CHmRun      CRuns      CWalks  DivisionW      PutOuts      Assists
##  2.3082206  0.9269527 -0.5968973 -120.5643234  0.1975004  0.5618678
##      Errors  NewLeagueN
## -7.1940771  71.8060203

predictors = c(names(c_adj2)[-c(1, 10, 14)], "Division", "NewLeague")
predictors

## [1] "AtBat"      "Hits"      "Walks"      "CAtBat"      "CHits"      "CHmRun"
## [7] "CRuns"      "CWalks"      "PutOuts"      "Assists"      "Errors"      "Division"

```



```

"
## [13] "NewLeague"

formula = reformulate(termlabels = predictors, response = "Salary")
formula

## Salary ~ AtBat + Hits + Walks + CAtBat + CHits + CHmRun + CRuns +
##      CWalks + PutOuts + Assists + Errors + Division + NewLeague

m1 = lm(formula, train_data)

y_h1 = predict(m1)
y_test1 = predict(m1, newdata = test_data)

train_mse1 = mean((train_data$Salary - y_h1)^2)
train_mse1

## [1] 88865.02

test_mse1 = mean((test_data$Salary - y_test1)^2)
test_mse1

## [1] 134736.7

```

Backward Subset selection according BIC

```

c_bic = coef(reg3, which.min(reg3_sum$bic))
c_bic

## (Intercept)      Walks      CAtBat      CHits      CHmRun  DivisionW
## 149.5853608    5.9582020   -0.5241611    2.0687418    1.7652811  -129.0165012

predictors = c(names(c_bic)[-c(1,6)], "Division")
predictors

## [1] "Walks"      "CAtBat"     "CHits"      "CHmRun"     "Division"

formula = reformulate(termlabels = predictors, response = "Salary")
formula

## Salary ~ Walks + CAtBat + CHits + CHmRun + Division

m2 = lm(formula, train_data)

y_h2 = predict(m2)
y_test2 = predict(m2, newdata = test_data)

train_mse2 = mean((train_data$Salary - y_h2)^2)
train_mse2

## [1] 97026.43

```

```
test_mse2 = mean((test_data$Salary - y_test2)^2)
test_mse2

## [1] 153090.5
```

Backward Subset selection according Mallow's C_p measure:

```
c_cp = coef(reg3,which.min(reg3_sum$cp))
c_cp

## (Intercept)      AtBat      Hits      Walks      CAtBat      CHits
## 151.3709120 -1.5393497  4.6939494  6.1305798 -0.3519984  1.1424820
##      CHmRun      CRuns      CWalks  DivisionW      PutOuts      Assists
##   2.3082206   0.9269527 -0.5968973 -120.5643234   0.1975004   0.5618678
##      Errors  NewLeagueN
##  -7.1940771  71.8060203

predictors = c(names(c_cp)[-c(1,10,14)],"Division","NewLeague")
predictors

## [1] "AtBat"      "Hits"      "Walks"      "CAtBat"      "CHits"      "CHmRun"
## [7] "CRuns"      "CWalks"    "PutOuts"    "Assists"    "Errors"     "Division"
##
## [13] "NewLeague"

formula = reformulate(termlabels = predictors, response = "Salary")
formula

## Salary ~ AtBat + Hits + Walks + CAtBat + CHits + CHmRun + CRuns +
##      CWalks + PutOuts + Assists + Errors + Division + NewLeague

m3 = lm(formula, train_data)

y_h3 = predict(m3)
y_test3 = predict(m3,newdata = test_data)

train_mse3 = mean((train_data$Salary - y_h3)^2)
train_mse3

## [1] 88865.02

test_mse3 = mean((test_data$Salary - y_test3)^2)
test_mse3

## [1] 134736.7
```

Comparison Matrix

```
Measure = c("Adjusted R sq.", "BIC", "Mallow's Cp")
Train_MSE_3 = c(train_mse1, train_mse2, train_mse3)
Test_MSE_3 = c(test_mse1, test_mse2, test_mse3)
```

#Comparison

```
output = data.frame(Measure,Train_MSE_3,Test_MSE_3)
output
```

```
##           Measure Train_MSE_3 Test_MSE_3
## 1 Adjusted R sq.    88865.02   134736.7
## 2           BIC    97026.43   153090.5
## 3 Mallow's Cp     88865.02   134736.7
```

7. Compare the results of the three methods in terms of choice of best model, train MSE and test MSE.

```
Method = rep(c("Best","Forward","Backward"),each= 3)
Measure_used = rep(Measure,times = 3)
Train_MSE = c(Train_MSE_1,Train_MSE_2,Train_MSE_3)
Test_MSE = c(Test_MSE_1,Test_MSE_2,Test_MSE_3)
compare_mse = data.frame(Method,Measure_used,Train_MSE,Test_MSE)
compare_mse
```

```
##      Method  Measure_used Train_MSE Test_MSE
## 1      Best Adjusted R sq.  88935.80 130066.9
## 2      Best           BIC  97026.43 153090.5
## 3      Best Mallow's Cp  89790.63 125652.3
## 4 Forward Adjusted R sq.  88287.08 134001.7
## 5 Forward           BIC  95658.81 146540.9
## 6 Forward Mallow's Cp  90888.44 145923.9
## 7 Backward Adjusted R sq.  88865.02 134736.7
## 8 Backward           BIC  97026.43 153090.5
## 9 Backward Mallow's Cp  88865.02 134736.7
```

Comparison of Training MSE:

- We observe that Adjusted R^2 gives the least training MSE in all three methods, of which forward subset method has the least. Thus we can say the forward method fits the data better than the others.

Comparison of Test MSE

- We observe that Mallow's C_p measure performs better than the other measures on an average, of which best Subset method has the least test MSE.

Interpretation

- Forward selection has the lowest training MSE, its test MSE is higher which indicates slight overfitting.
- Backward selection performs similarly to forward but slightly worse.
- Best subset selection, especially using C_p , provides the best balance between bias and variance.

Conclusion

-Evaluating all possible models (best subset) can lead to better generalization. -Stepwise methods (forward/backward) are faster but may not always give the optimal model, as some useful combinations might be left out.

Problem Set 7: Shrinkage Methods

We would be working with the Hitters dataset available in the ISLR library. We have to predict the Salary based on all available variables. First, we make sure that we delete all rows with one or more missing values.

1. We split the data set into a training set and a test set randomly, keeping 100 observations in the test set and the remaining in the training set.

```
set.seed(123)
rm(list=ls())
library(glmnet)
library(ISLR)

Hitters = na.omit(Hitters)

test_index = sample(1:nrow(Hitters), 100)

test_data = Hitters[test_index, ]
train_data = Hitters[-test_index, ]
```

2. We train the model using least squares and report the train MSE and test MSE.

```
lm_model = lm(Salary ~ ., data = train_data)
summary(lm_model)
```

```
##
## Call:
## lm(formula = Salary ~ ., data = train_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -809.76 -182.13  -54.69   138.36 1837.14
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  212.67449   129.87001    1.638  0.103705
## AtBat        -2.04895     0.87954   -2.330  0.021229 *
## Hits         5.85495     3.15579    1.855  0.065612 .
## HmRun        5.43622     9.22155    0.590  0.556448
## Runs        -2.49915     4.03131   -0.620  0.536288
## RBI          2.32756     3.84573    0.605  0.545984
## Walks        6.40986     2.41415    2.655  0.008827 **
```

```
## Years      -8.30051    17.14327   -0.484  0.628996
## CAtBat     0.01569     0.19354    0.081  0.935512
## CHits      -0.54632     1.02039   -0.535  0.593204
## CHmRun     -2.38189     2.49628   -0.954  0.341604
## CRuns       1.56320     1.05024    1.488  0.138840
## CRBI        1.26993     1.02979    1.233  0.219524
## CWalks     -0.95946     0.45825   -2.094  0.038049 *
## LeagueN    44.32184    98.13766    0.452  0.652220
## DivisionW  -149.54532   56.87996   -2.629  0.009496 **
## PutOuts     0.41589     0.11170    3.723  0.000282 ***
## Assists     0.43589     0.28787    1.514  0.132176
## Errors     -5.50162     5.83866   -0.942  0.347642
## NewLeagueN -15.79562    97.46148   -0.162  0.871479
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 346.5 on 143 degrees of freedom
## Multiple R-squared:  0.4782, Adjusted R-squared:  0.4088
## F-statistic: 6.897 on 19 and 143 DF,  p-value: 1.144e-12

y_hat = predict(lm_model, newdata = train_data)
y_test = predict(lm_model, newdata = test_data)

train_mse = mean((train_data$Salary-y_hat)^2)
train_mse

## [1] 105334.2

test_mse = mean((test_data$Salary-y_test)^2)
test_mse

## [1] 87803.91
```

3. We fit ridge regression models on the training set, with grid values of λ . We report the following:

- (i) the dimension of the coefficient matrix so obtained.
- (ii) The l_2 norm of the regression coefficients corresponding to the 50th and 60th choice of λ . We comment on the output.
- (iii) Test MSE corresponding to these two choices of λ .

```
lambda_grid = 10^seq(10, -2, length = 100)
x_train = model.matrix(Salary~.,train_data)[,-1]
x_test = model.matrix(Salary~.,test_data)[,-1]
head(x_train)

##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun
## -Alan Ashby    315   81     7   24  38   39   14   3449   835    69
## -Alvin Davis   479  130    18   66  72   76    3   1624   457    63
## -Andre Dawson  496  141    20   65  78   37   11   5628  1575   225
## -Alfredo Griffin 594  169     4   74  51   35   11   4408  1133    19
## -Andres Thomas 323   81     6   26  32    8    2    341    86     6
```

```
## -Andre Thornton      401   92   17   49   66   65   13   5206   1332   253
##                      CRuns CRBI CWalks LeagueN DivisionW PutOuts Assists Errors
## -Alan Ashby          321  414   375         1         1    632    43    10
## -Alvin Davis         224  266   263         0         1    880    82    14
## -Andre Dawson        828  838   354         1         0    200    11     3
## -Alfredo Griffin     501  336   194         0         1    282   421    25
## -Andres Thomas       32   34     8         1         1   143   290    19
## -Andre Thornton      784  890   866         0         0     0     0     0
##                      NewLeagueN
## -Alan Ashby          1
## -Alvin Davis         0
## -Andre Dawson        1
## -Alfredo Griffin     0
## -Andres Thomas       1
## -Andre Thornton      0

ridge_model=glmnet(x_train, train_data$Salary, alpha = 0,lambda=lambda_grid)
dim(coef(ridge_model))

## [1]  20 100
```

(i) Dimension of Coefficient Matrix

```
dim(coef(ridge_model))

## [1]  20 100
```

(ii) Coefficients for 50th and 60th value of λ respectively

```
# 50th value
coef_50 = coef(ridge_model)[, 50]
coef_50

## (Intercept)      AtBat      Hits      HmRun      Runs
## 438.509969440  0.029684297  0.110523230  0.560867468  0.178502584
##      RBI      Walks      Years      CAtBat      CHits
## 0.252163513  0.238337160  1.002466622  0.002796464  0.010194612
##      CHmRun      CRuns      CRBI      CWalks      LeagueN
## 0.077846378  0.020089803  0.021957346  0.021866228  0.432252998
##      DivisionW      PutOuts      Assists      Errors      NewLeagueN
## -6.916280599  0.018294166  0.002998934 -0.082552703  0.741567242

cat("\n")

#60th Value
coef_60 = coef(ridge_model)[, 60]
coef_60

## (Intercept)      AtBat      Hits      HmRun      Runs      RBI
## 105.63996577  0.08310078  0.51246876  2.09635144  0.60993278  1.14623958
##      Walks      Years      CAtBat      CHits      CHmRun      CRuns
## 1.01677160  2.77366963  0.01043635  0.04276159  0.27032634  0.08012067
##      CRBI      CWalks      LeagueN      DivisionW      PutOuts      Assists
## 0.09127173  0.05840151  18.35350369 -62.46410535  0.14784491  0.04002532
```

```
##      Errors    NewLeagueN
## -0.86723778  12.21693330
```

(ii). L2 Norm of Coefficients

```
l2_norm = function(beta){
  sqrt(sum(beta^2))
}
```

```
# L2 norm of 50th Lambda
```

```
l2_50 = l2_norm(coef_50)
```

```
l2_50
```

```
## [1] 438.5671
```

```
# L2 norm of 60th Lambda
```

```
l2_60 = l2_norm(coef_60)
```

```
l2_60
```

```
## [1] 124.7542
```

(iii). Test MSE for 50th and 60th λ

```
# Predictions
```

```
ridge_pred_50 = predict(ridge_model, s = lambda_grid[50], newx = x_test)
```

```
ridge_pred_60 = predict(ridge_model, s = lambda_grid[60], newx = x_test)
```

```
# Test MSE for 50th Lambda
```

```
Test_mse_50 = mean((test_data$Salary - ridge_pred_50)^2)
```

```
Test_mse_50
```

```
## [1] 167303.1
```

```
# Test MSE for the 60th Lambda
```

```
Test_mse_60 = mean((test_data$Salary - ridge_pred_60)^2)
```

```
Test_mse_60
```

```
## [1] 97935.56
```

We observe that the test MSE is dropping as Lamda increases.